

Deploying Application using Docker-Compose

Name: Pooja Prakash Belgaumkar

- Docker Compose helps automate the creation and management of multiple images and containers for a microservices-based application.
- It is written using **YAML (.yml or .yaml)** files.
- With Docker Compose, there is no need to build images and create containers separately for each service. All the images and containers can be started together using a single command, **docker compose up**, and stopped together using the **docker compose down** command.

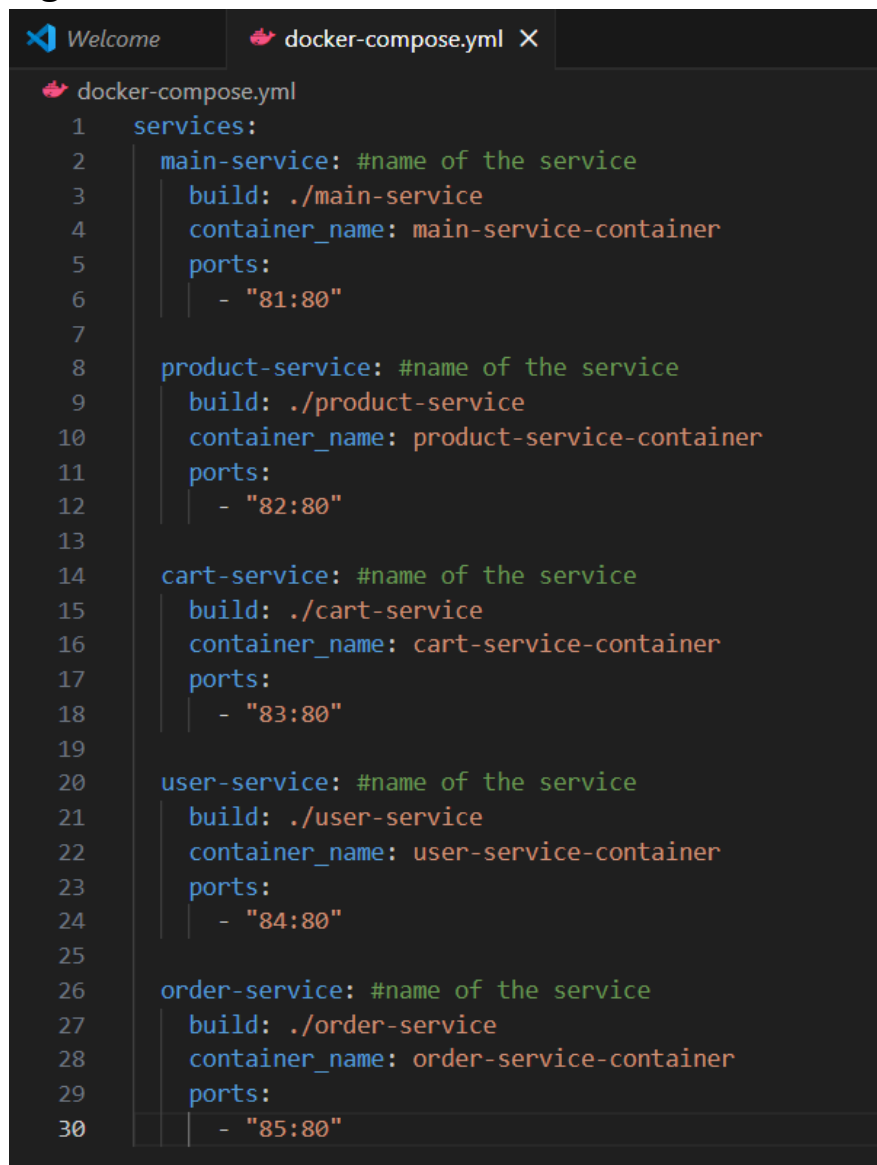
1. First, we clone the application repository from **GitHub** to the server.

```
[root@ip-172-31-15-198 ec2-user]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-15-198 ec2-user]# git --version
git version 2.50.1
[root@ip-172-31-15-198 ec2-user]# git clone https://github.com/PoojaPrakash08/Kastro-Ecom-DC
Cloning into 'Kastro-Ecom-DC'...
remote: Enumerating objects: 42, done.
remote: Counting objects: 100% (42/42), done.
remote: Compressing objects: 100% (35/35), done.
remote: Total 42 (delta 9), reused 32 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (42/42), 49.11 KiB | 4.91 MiB/s, done.
Resolving deltas: 100% (9/9), done.
[root@ip-172-31-15-198 ec2-user]# ls
ECR-ECS-Project  Kastro-Ecom-DC
[root@ip-172-31-15-198 ec2-user]# cd Kastro-Ecom-DC
[root@ip-172-31-15-198 Kastro-Ecom-DC]# ls
README.md          main-service        postcss.config.js   tsconfig.json
cart-service       nginx.conf          product-service     tsconfig.node.json
docker-compose.yml order-service       src                 user-service
eslint.config.js   package-lock.json  tailwind.config.js  vite.config.ts
index.html         package.json        tsconfig.app.json
```

2. Then, we install **Docker Compose** to manage and run the microservices-based application.

```
[root@ip-172-31-15-198 Kastro-Ecom-DC]# sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL \
https://github.com/docker/compose/releases/download/v2.29.2/docker-compose-linux-x86_64 \
-o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             %         Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0    0     0      0     0  --:--:-- --:--:-- --:--:--    0
100 60.2M 100 60.2M    0     0 53.2M    0  0:00:01  0:00:01 --:--:-- 73.2M
[root@ip-172-31-15-198 Kastro-Ecom-DC]# docker compose version
Docker Compose version v2.29.2
[root@ip-172-31-15-198 Kastro-Ecom-DC]#
```

3. Next, we create a **Docker Compose** file that includes all the required services, along with details such as **image names**, **container names**, and **port mappings**.



```
docker-compose.yml
1  services:
2    main-service: #name of the service
3      build: ./main-service
4      container_name: main-service-container
5      ports:
6        - "81:80"
7
8    product-service: #name of the service
9      build: ./product-service
10     container_name: product-service-container
11     ports:
12       - "82:80"
13
14    cart-service: #name of the service
15      build: ./cart-service
16      container_name: cart-service-container
17      ports:
18        - "83:80"
19
20    user-service: #name of the service
21      build: ./user-service
22      container_name: user-service-container
23      ports:
24        - "84:80"
25
26    order-service: #name of the service
27      build: ./order-service
28      container_name: order-service-container
29      ports:
30        - "85:80"
```

4. After writing the Docker Compose file, we build and start the images and containers using the Docker Compose commands.

```
[root@ip-172-31-15-198 Kastro-Ecom-DC]# vi docker-compose.yml
[root@ip-172-31-15-198 Kastro-Ecom-DC]# docker images
REPOSITORY      TAG          IMAGE ID       CREATED        SIZE
[root@ip-172-31-15-198 Kastro-Ecom-DC]# docker ps
CONTAINER ID    IMAGE                COMMAND        CREATED        STATUS        PORTS          NAMES
[root@ip-172-31-15-198 Kastro-Ecom-DC]# docker compose up -d --build
[+] Building 24.5s (35/35) FINISHED                                docker:default
=> [user-service internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 232B                                                            0.0s
=> [product-service internal] load build definition from Dockerfile                            0.1s
=> => transferring dockerfile: 232B                                                            0.0s
=> [order-service internal] load build definition from Dockerfile                            0.1s
=> => transferring dockerfile: 232B                                                            0.0s
=> [product-service internal] load metadata for docker.io/library/ubuntu:latest              1.6s
=> [main-service internal] load build definition from Dockerfile                            0.0s
=> => transferring dockerfile: 232B                                                            0.0s
```

```
[root@ip-172-31-15-198 Kastro-Ecom-DC]# docker images
REPOSITORY      TAG          IMAGE ID       CREATED        SIZE
kastro-ecom-dc-user-service    latest      7f4151128d8b   About a minute ago   243MB
kastro-ecom-dc-main-service    latest      9ee86f94ffae   About a minute ago   243MB
kastro-ecom-dc-product-service latest      8e01b3122fa7   About a minute ago   243MB
kastro-ecom-dc-order-service   latest      560590c207d3   About a minute ago   243MB
kastro-ecom-dc-cart-service    latest      dce7f9d2d8cd   About a minute ago   243MB
[root@ip-172-31-15-198 Kastro-Ecom-DC]# docker ps
CONTAINER ID    IMAGE                COMMAND        NAMES
STATUS          PORTS          CREATED
b551ad80f3d5    kastro-ecom-dc-order-service   "/usr/sbin/apachectl..."   About a minute ago
Up About a minute 0.0.0.0:85->80/tcp, :::85->80/tcp order-service-container
f8e353c818b1    kastro-ecom-dc-user-service    "/usr/sbin/apachectl..."   About a minute ago
Up About a minute 0.0.0.0:84->80/tcp, :::84->80/tcp user-service-container
22399d2aebd7    kastro-ecom-dc-main-service    "/usr/sbin/apachectl..."   About a minute ago
Up About a minute 0.0.0.0:81->80/tcp, :::81->80/tcp main-service-container
7c3fde4032c4    kastro-ecom-dc-product-service "/usr/sbin/apachectl..."   About a minute ago
Up About a minute 0.0.0.0:82->80/tcp, :::82->80/tcp product-service-container
68006fb6f230    kastro-ecom-dc-cart-service    "/usr/sbin/apachectl..."   About a minute ago
Up About a minute 0.0.0.0:83->80/tcp, :::83->80/tcp cart-service-container
[root@ip-172-31-15-198 Kastro-Ecom-DC]#
```

5. To access the web pages, we configure the **inbound rules** of the instance to allow access to the required **port range**.

☰ [EC2](#) > [Security Groups](#) > [sg-0db2eed90a7b53d7e - launch-wizard-17](#) > Edit inbound rules ⓘ 🖨️ ⌂

Description - optional ⓘ

Inbound rule 4 Delete

Security group rule ID	Type ⓘ	Protocol ⓘ
–	Custom TCP ▼	TCP
Port range ⓘ	Source type ⓘ	Source ⓘ
81 - 85	Anywhere-IPv4 ▼	0.0.0.0/0 X
Description - optional ⓘ		

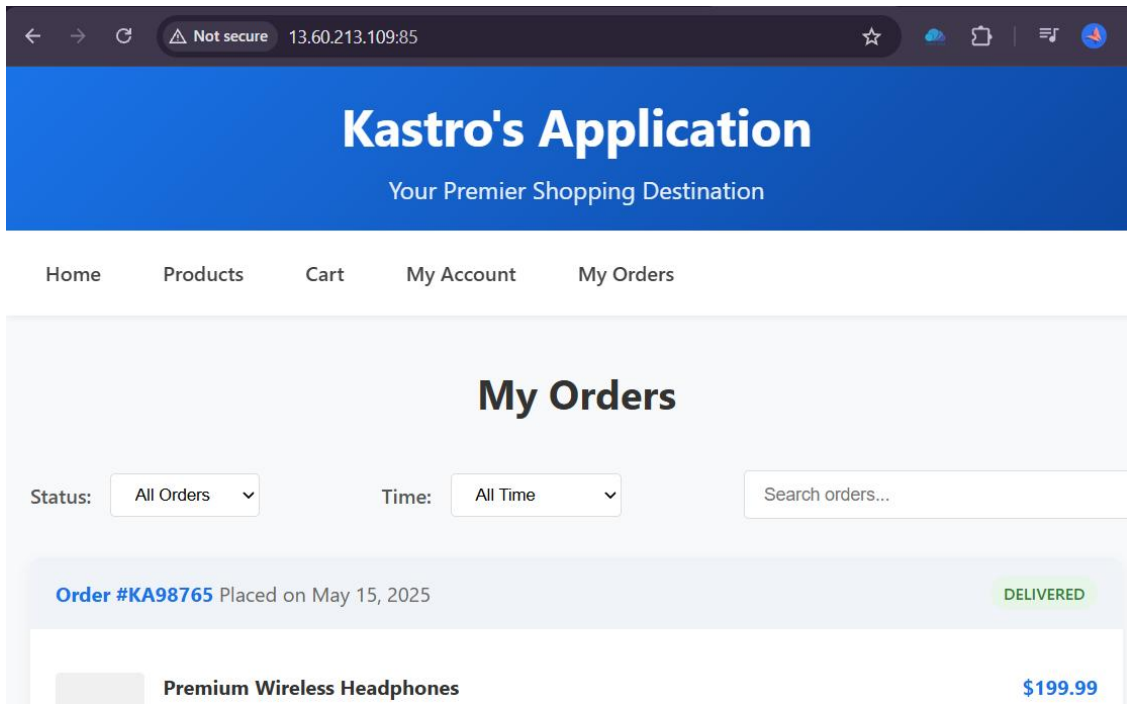
Add rule

⚠️ Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only. X

Cancel Preview changes Save rules

6. When we access the application using the allowed port numbers, each web service page is displayed on its corresponding port.





7. Finally, we can stop and remove all the running containers and images using the **docker compose down** command.

```
[root@ip-172-31-15-198 Kastro-Ecom-DC]# docker compose down
[+] Running 6/6
 ✓ Container user-service-container      Remo...      11.0s
 ✓ Container order-service-container     Remo...      11.3s
 ✓ Container main-service-container      Remo...      11.2s
 ✓ Container product-service-container   R...        10.7s
 ✓ Container cart-service-container      Remo...      10.6s
 ✓ Network kastro-ecom-dc_default        Remove...    0.6s
[root@ip-172-31-15-198 Kastro-Ecom-DC]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
kastro-ecom-dc-user-service   latest     7f4151128d8b  10 minutes ago 243MB
kastro-ecom-dc-main-service   latest     9ee86f94ffae  10 minutes ago 243MB
kastro-ecom-dc-product-service latest     8e01b3122fa7  10 minutes ago 243MB
kastro-ecom-dc-order-service  latest     560590c207d3  10 minutes ago 243MB
kastro-ecom-dc-cart-service   latest     dce7f9d2d8cd  10 minutes ago 243MB
[root@ip-172-31-15-198 Kastro-Ecom-DC]# docker ps
CONTAINER ID   IMAGE          COMMAND        CREATED        STATUS        PORTS        NAMES
[root@ip-172-31-15-198 Kastro-Ecom-DC]#
```